

```

2. int main(int argc, char *argv[])
{
    int n; pid_t pid;
    n = strtoll(argv[1], NULL, 10);
    if((pid = fork()) < 0)
    {
        perror("Error en fork");
        exit(-1);
    }
    else if (pid == 0)
    {
        if (n % 2 == 0) printf("Par");
        else printf("Impar");
    }
    else
    {
        if (n % 4 == 0) printf("Divisible por 4");
        else printf("No divisible por 4");
    }
    exit(0);
}

```

```

4. int main(int argc, char *argv[]) {
    pid_t pid;
    for(int i=0; i<5; i++){
        if((pid = fork()) == -1) { perror("Error en fork"); exit(EXIT_FAILURE); }
        if(pid == 0) { printf("soy el hijo %d\n", (int) getpid());
            exit(0); }
    }
    int hijos = 5; int status; pid_t hijo;
    while(hijos > 0){
        hijo = wait(&status); hijos--;
        printf("Acaba de finalizar mi hijo con %d, me quedan %d hijos vivos\n", (int) hijo, hijos);
    }
    exit(0)
}

```

```

5. int main (int argc, char* argv[]) {
    pid_t pid; pid_t pids[5];
    for (int i=0; i<5; i++) {
        if ((pid = fork()) == -1) { perror("Error en fork"); exit(EXIT_FAILURE); }
        if (pid == 0) { printf("soy el hijo %.d\n", (int) getpid()); exit(0); }
        pids[i] = pid;
    }

    int hijos = 5; int status; pid_t hijo;
    for (int i=0; i<5; i+=2) {
        hijo = waitpid(pids[i], &status, 0); hijos--;
        printf("Acaba de finalizar mi hijo con %.d, me quedan  
%.d hijos vivos\n", (int) hijo, hijos);
    }

    for (int i=1; i<5; i+=2) {
        hijo = waitpid(pids[i], &status, 0); hijos--;
        printf("Acaba de finalizar mi hijo con %.d, me quedan  
%.d hijos vivos\n", (int) hijo, hijos);
    }
    exit(0);
}

```

```

7. int main (int argc, char * argv[]) {
    pid_t pid; int status;
    bool bg = (strcmp(argv[argc-1], "bg") == 0);
    if (pid = fork()) < 0 {
        perror("Error en fork")
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        if (bg) argv[argc-1] = NULL;
        execvp(argv[1], &argv[1]);
        exit(EXIT_FAILURE);
    }
    else if (pid > 0) {
        if (! bg)
            waitpid(pid, &status, 0);
        exit(0);
    }
}

```

(PS) 2

```
static int contador[NSIG];
```

```
static void handler(int sigNum){
```

```
    contador[sigNum]++;
```

```
    printf("La señal %d se ha recibido %d veces", sigNum,
```

```
    contador[sigNum]);
```

```
int main(int argc, char *argv[]){
```

```
    struct sigaction action;
```

```
    printf("No puedo manejar la señal %d", SIGKILL);
```

```
    printf("No puedo manejar la señal %d", SIGSTOP);
```

```
    action.sa_handler = handler;
```

```
    sigemptyset(&action.sa_mask);
```

```
    action.sa_flags = 0;
```

```
    for(int i=1; i<NSIG; i++){
```

```
        if(i == SIGKILL || i == SIGSTOP) continue;
```

```
        sigaction(i, &action, NULL);
```

```
    }
```

```
    for(i=1; i<NSIG; i++){
```

```
    }
```

(P5) 3.

```
static bool recibido = false;
```

```
static void handler(int sig){ recibido = true; }
```

```
int main(int argc, char *argv[]){
```

```
    struct sigaction action;
```

```
    sigset_t block_usr1, oldmask, suspend_mask;
```

```
    handler { action.sa_handler = handler;
para SIGUSR1 { sigemptyset(&action.sa_mask);
                action.sa_flags = 0;
                sigaction(SIGUSR1, &action, NULL);
```

```
    bloque de { sigemptyset(&block_usr1);
Señales      { sigaddset(&block_usr1, SIGUSR1);
solo con     { sigprocmask(SIG_BLOCK, &block_usr1, &oldmask);
SIGUSR1
```

```
    bloque de { sigfillset(&suspend_mask);
todas las    { sigdelset(&suspend_mask, SIGUSR1);
señales      { while(!recibido) sigsuspend(&suspend_mask);
menos        SIGUSR1
```

```
    destablogear { sigprocmask(SIG_SETMASK, &oldmask, NULL);
proceso        { exit(0);
                }
```

```

1.2. int main(int argc, char * argv[]) {
    int fd; int salida; int leidos; char buffer[80];
    int bloque = 0; char header[20];

    if ((fd = open(argv[1], O_RDONLY)) < 0) {
        perror("Error al leer archivo");
        exit(-1);
    }

    if ((salida = open("salida.txt", O_CREAT | O_TRUNC |
        O_WRONLY, S_IRUSR | S_IWUSR)) < 0) {
        perror("Error al crear archivo de salida");
        exit(-1);
    }

    while ((leidos = read(fd, buffer, 80)) > 0) {
        int len = sprintf(header, sizeof(header),
            "Bloque %d", bloque++);

        if (write(salida, buffer, leidos) < 0) {
            perror("Error de escritura");
            exit(-1);
        }

        write(salida, "\n", 2);
    }

    close(fd); close(salida);
    exit(0)
}

```

```

2.2 int main( int argc, char * argv[]){
    DIR * dir; struct dirent *entry; char ruta [1024];

    if((dir = opendir( argv[1] )) < 0){
        perror(" Error al abrir directorio");
        exit(-1);
    }

    while ((entry = readdir (dir)) != NULL) {
        if ((strcmp(".", entry->d_name) == 0) ||
            strcmp("..", entry->d_name) == 0)
            continue;

        sprintf(ruta, sizeof(ruta), "%s %s",
                argv[1], entry->d_name);

        struct stat atributos;
        if( stat(ruta, &atributos) < 0){
            perror(" Error al leer
                    metadatos");
            exit(-1);
        }

        mode_t permisos = strtol( argv[2],
                                   NULL, 8);

        int resultado = chmod(ruta, permisos);
        if resultado (< 0){
            perror(" Error al cambiar permisos");
            exit(-1);
        }
    }
}

```

```

2.3. void recorrer (const char * path, int * tam, int * contador){
    DIR * dir; struct dirent * entrada;
    char [1024] ruta; struct stat atributos;

    if ((dir = opendir(argv[1])) < 0){
        perror("Error al leer directorio"); exit(-1);
    }

    while ((entrada = readdir(dir)) != NULL){
        if ((strcmp(entrada->d_name) == 0) ||
            (strcmp(entrada->d_name) == ".")) continue;
        sprintf(ruta, sizeof(ruta), "%s/%s", argv[1], entrada->d_name);

        if (stat(ruta, &atributos) < 0){
            perror("Error al obtener metadatos"); exit(-1);
        }

        if (S_ISREG(atributos.st_mode)){
            *espacio += atributos.st_size;
            (*contador)++;
            printf("metadatos.st_ino");
        }
        else if (S_ISDIR(atributos.st_mode))
            recorrer(ruta, tam, contador);
    }
    closedir(dir);
}

int main(int argc, char * argv[]){
    int tam = 0, contador = 0;
    recorrer(argv[1], tam, contador);
    exit(0);
}

```



```
3.1. int main(int argc, char * argv[]){  
    pid_t pid; int n = argv[1];  
    if(argc < 2){ perror("Faltan argumentos"); exit(-1);}  
    if((pid = fork()) < 0){ perror("Error en fork"); exit(-1);}  
    if(pid == 0){  
        if(n % 2 == 0) printf("Número par");  
        else printf("Número impar");  
    }  
    else{  
        if(n % 4 == 0) printf("Número divisible por 4");  
        else printf("Número no divisible por 4");  
    }  
    wait(NULL);  
}
```

3.4. `int main(int argc, char * argv[]) {`

`pid_t pid; int hijos = 5; int status;`

`for(int i = 1; i < 5; i++) {`

`if ((pid = fork()) < 0) { perror("Error en fork"); exit(-1); }`

`if (pid == 0) {`

`printf("Soy el hijo %d", (int) getpid()); exit(0)`

`}`

`}`

`while(hijos > 0) {`

`pid_t hijo = wait(NULL); hijos --;`

`printf("Acaba de finalizar mi hijo con PID %d,`

`quedan %d hijos vivos, (int) getpid(), hijos);`

`}`

`exit(0);`

`}`

3.5. `int main(int argc, char * argv[]) {`

`pid_t pid, hijo; int hijos; pid_t pids[5];`

`for(int i = 1; i < 5; i++) {`

`if ((pid = fork()) < 0) { perror("Error en fork"); exit(-1); }`

`if (pid == 0) { printf("Soy el hijo %d", (int) getpid()); exit(0); }`

`pids[i] = pid;`

`}`

`for(int i = 1; i <= 5; i++) {`

`hijo = waitpid(pids[i], NULL, 0); hijos --;`

`printf("Acaba de finalizar mi hijo con PID %d,`

`quedan %d hijos vivos, (int) getpid(), hijos);`

`}`

`for(int i = 1; i <= 5; i++) {`

`hijo = waitpid(pids[i], NULL, 0); hijos --;`

`printf("Acaba de finalizar mi hijo con PID %d,`

`quedan %d hijos vivos, (int) getpid(), hijos);`

`}`

`exit(0);`

`}`

```
3.7. int main (int argc, char * argv[]) {
```

```
    char * programa = argv[1]; bool bg;
```

```
    bg = (strcmp ("bg", argv[argc-1]) == 0);
```

```
    if (bg) { argv[argc-1] = NULL; argc --; }
```

```
    if ((pid = fork()) < 0) { printf("Error en fork"); exit(1-1); }
```

```
    if (pid == 0) { if (bg) execvp(programa, &programa); exit(0); }
```

```
    else if (pid > 0)
```

```
        if (!bg) wait(NULL);
```

```
    exit(0);
```

```
}
```

4.5.

esclavo.c

```
int primo(int n){  
    if (n < 2) return 0;  
    else if (n == 2) return 1;  
    else if (n % 2 == 0) return 0;  
  
    int top = (int) sqrt((double)n);  
    for (int i = 3; i <= top; i += 2)  
        if (n % i == 0) return 0;  
  
    return 1;  
}
```

```
int main(int argc, char * argv[]){  
    int inf = atoi(argv[1]);  
    int sup = atoi(argv[2]);  
  
    for (int i = inf; i <= sup; i++){  
        if (primo){  
            int p = n; write(STDOUT_FILENO, &p, sizeof(int));  
        }  
    }  
  
    exit(0);  
}
```

maestro.c

```
void lanzar-esclavo(int fd, int inf, int sup){  
    dup2l(fd, STDOUT_FILENO); char a[32], b[32];  
    sprintf(a, sizeof(a), "%d", inf);  
    sprintf(b, sizeof(b), "%d", sup);  
    execl("./esclavo", "esclavo", a, b, (char*)NULL, _exit(-1))  
}
```

```
int main(int argc, char* argv[]){  
    int inf = atoi(argv[1]); int sup = atoi(argv[2]);  
    int mid = (inf + sup) / 2; int p1[2], p2[2]; int x;  
    if(pipe(p1) < 0){ perror("Error"); exit(-1);}  
    if(pipe(p2) < 0){ perror("Error"); exit(-1);}  
    pid_t c1 = fork(), c2 = fork();  
    if(c1 < 0){ perror("Error en fork"); exit(-1);}  
    if(c1 == 0){  
        close(p1[0]); close(p2[0]); close(p2[1]);  
        lanzar-esclavo(p1[1], inf, mid);  
    }  
    if(c2 < 0){ perror("Error en fork"); exit(-1);}  
    if(c2 == 0){  
        close(p2[0]); close(p1[0]); close(p1[1]);  
        lanzar-esclavo(p2[1], mid, sup);  
    }  
    while((read(p1[0], &x, sizeof(int)) == sizeof(int))  
        printf("%d", x);  
    while((read(p2[0], &x, sizeof(int)) == sizeof(int))  
        printf("%d", x);  
    close(p1[0]); close(p2[0]);  
    waitpid(c1, NULL); waitpid(c2, NULL);  
    return 0;  
}
```

5.2.

```
static int contador[64] = {0};  
static void handler(int sig){  
    contador[sig]++;  
    printf("La señal %d ha sido recibida %d veces, sig: %d, contador[%d];",  
sig, sig, contador[sig]);  
}
```

```
int main(int argc, char * argv[]) {  
    struct sigaction action;  
    printf("No puedo manejar la señal %d, SIGKILL);  
    printf("No puedo manejar la señal %d, SIGSTOP);  
    action.sa_handler = handler;  
    sigemptyset(&action.sa_mask);  
    action.sa_flags = 0;  
    for(i = 1; i < 64; i++){  
        if(i == SIGKILL || i == SIGSTOP) continue;  
        else if(sigaction(i, &action, NULL) < 0){  
            perror("Error en el manejador");  
        }  
    }  
    for(;;);  
}
```